MACHINE LEARNING OPERATIONS

Assignment 01
End-to-End Heart Disease ML System

Data acquisition | *Experiments* | *CI/CD* | *Docker* | *API* | *Monitoring*

Item	Implementation
Dataset	UCI Heart Disease - processed Cleveland subset
Target	Binary: original 0 -> absence; original 1-4 -> presence
Models	Logistic Regression and Random Forest
Experiment tracking	MLflow runs, params, metrics, CV tables, plots, model artifacts
Serving / monitoring	FastAPI + Docker + Prometheus + Grafana; Kubernetes manifests

Executive summary

This project applies a reproducible MLOps workflow to a small historical tabular dataset. A downloader extracts the official UCI file, a cleaning step normalizes missing-value markers and derives the binary target, and an EDA script generates class-balance, feature-distribution, missingness, and correlation plots. Two scikit-learn pipelines are tuned using stratified cross-validation and evaluated on a held-out test set. MLflow tracks experiments, while FastAPI, Docker, CI, Kubernetes manifests, request logs, Prometheus metrics, and a provisioned Grafana dashboard cover delivery and operational visibility.

Important: all reported performance values must come from a successful run on the downloaded dataset. This report generator inserts values from reports/metrics.json when available; it does not manufacture metrics when the external dataset has not been downloaded.

1. Problem, Dataset, and Assumptions

The assignment asks for a binary classifier that consumes patient health features and predicts whether heart disease is present, then deploys the model as a cloud-ready monitored API. This is implemented as a coursework demonstration, not a clinically validated prediction system.

Dataset source and schema

The official UCI Heart Disease dataset page describes four source databases and 303 instances in the processed Cleveland subset. It also explains that the original goal field takes values 0 to 4 and that published Cleveland experiments commonly distinguish 0 (absence) from 1-4 (presence). The implementation uses the no-header processed Cleveland file from the UCI archive and declares 13 predictor columns plus the original goal field.

Decision	Reason / consequence
Cleveland subset	Keeps to the classic 303-record processed file, suitable for this assignment's scope.
Binary target	Maps original goal 0 to 0 and any positive goal to 1.
Missing features	`?` is normalized to NaN; imputation is fit inside CV/training pipelines.
Train/test protocol	Stratified 80/20 split, random_state=42; test set is not used for tuning.
Model selection	5-fold StratifiedKFold; mean validation ROC-AUC is the selection metric.
Reproducibility	A fixed random seed, parameter grids, requirements file, and data hash manifest.

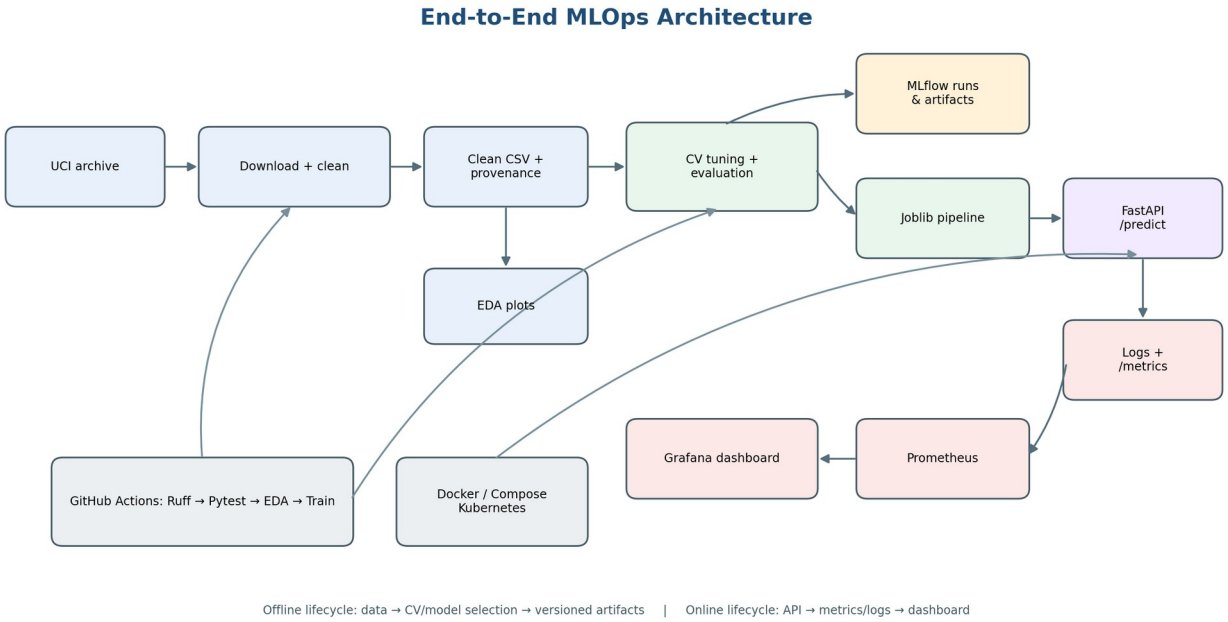
Known limitations

- Historical, small dataset; the split metrics have high sampling uncertainty and are not evidence of clinical utility.
- Feature names and coded values need domain context; input validation enforces basic ranges but is not medical validation.
- The API intentionally returns an educational model estimate and a clear not-for-diagnosis disclaimer.

2. System Architecture

Data plane, model lifecycle, and delivery path

The workflow separates offline model development from online inference. The raw archive is normalized to a stable tabular schema, and preprocessing stays attached to each estimator so the serving service can load one serializable object and apply the same transformations that were fit during training.



Principal components

Component	Responsibility
Data/EDA	Fetch, clean, record provenance, summarize missingness and distributions.
Training	Grid search for two pipelines, CV and holdout metrics, plots and model selection.
MLflow	Model-family runs with parameters, metrics, cross-validation tables, evaluation plots and model artifacts.
Serving	FastAPI schema validation, prediction, probability, health/readiness routes and safe request logs.
Operations	Docker image, Compose-based Prometheus/Grafana stack, Kubernetes Deployment/Service.
CI	Ruff, pytest, data acquisition, EDA, training, and artifact upload on GitHub Actions.

3. Data Acquisition, Cleaning, and EDA

Run `python -m heart_mlops.data --output data/processed/heart_disease_clean.csv`. The downloader retrieves the UCI ZIP archive, locates the processed Cleveland file, extracts the raw no-header CSV, writes a named cleaned dataset, and saves a JSON provenance manifest. The manifest records source URL, raw-file SHA-256, row count, feature count, missing feature cells, and class counts.

Cleaning rules

- Use an explicit 14-column schema rather than relying on column inference.
- Normalize `?` markers and invalid numeric strings to missing values.
- Drop rows with missing targets; derive `target = 1` when original `num > 0`, otherwise 0.
- Drop exact duplicate rows and persist features in a fixed order; preserve feature NaNs for train-fold imputation.

EDA outputs

Plot	Question answered
Numeric histograms by target	How do the five continuous features distribute by class?
Target balance	How many records map to absence vs presence?
Numeric correlation heatmap	What pairwise linear relationships appear among continuous measurements?
Missingness bars	Which feature fields need imputation and how many cells are missing?

Execute `python -m heart_mlops.eda --input data/processed/heart_disease_clean.csv --output reports/figures`. Numeric-only correlation is chosen to avoid presenting ordinal/coded categorical values as continuous measurements. Interpret the plot as exploratory association, not causation.

Observed values are generated in `reports/eda_summary.json`; do not copy general expected class ratios into the submission. Inspect the images after download and write two or three dataset-specific observations based on the actual output.

4. Feature Engineering and Model Development

The 13 fields are separated into five continuous/numeric measurements (age, trestbps, chol, thalach, oldpeak) and eight coded categorical fields (sex, cp, fbs, restecg, exang, slope, ca, thal). The data are not globally scaled or imputed before splitting. Instead, each model contains a ColumnTransformer fitted only on training folds.

Step	Logistic Regression	Random Forest
Numeric missing values	Median imputation	Median imputation
Categorical missing values	Most-frequent imputation	Most-frequent imputation
Categorical encoding	OneHotEncoder, unknown categories ignored	OneHotEncoder, unknown categories ignored
Numeric scaling	StandardScaler	No scaling
Hyperparameter grid	C in {0.1, 1, 10}; class_weight in {None, balanced}	n_estimators in {100, 250}; max_depth in {None, 6}; min_samples_leaf in {1, 3}; class_weight in {None, balanced}

Protocol

- Use a stratified 80/20 train/test split with random_state=42.
- Tune using 5-fold StratifiedKFold with shuffle=True and random_state=42.
- Score candidate configurations with ROC-AUC, accuracy, precision, and recall; refit the best mean ROC-AUC configuration.
- Report holdout accuracy, precision, recall, F1, and ROC-AUC plus the confusion matrix, ROC curve, classification report, and transformed feature importance.
- Select the model family by cross-validation ROC-AUC, not by searching for the best-looking holdout score.

ROC-AUC captures ranking quality across thresholds; precision describes the fraction of positive predictions that are correct; recall describes positive-class coverage. These metrics can conflict, so results should be discussed in context and not reduced to accuracy alone.

5. Experiment Results and Model Selection

Results pending execution. The data archive could not be downloaded in the report-generation environment, so no scores are guessed or presented as if measured. After the UCI download succeeds, run the training command in the README and execute `python scripts/generate_report.py` again. The regenerated table will be filled from `reports/metrics.json`.

Model	CV ROC-AUC	Test accuracy	Test precision	Test recall	Test ROC-AUC
Logistic Regression	Run pipeline	Run pipeline	Run pipeline	Run pipeline	Run pipeline
Random Forest	Run pipeline	Run pipeline	Run pipeline	Run pipeline	Run pipeline

How to interpret the result

- Compare cross-validation mean and standard deviation to assess both average quality and fold sensitivity.
- Compare holdout precision and recall to show the practical trade-off for classifying the positive class.
- Use the confusion matrix to identify false positives and false negatives; do not infer medical harm or benefit from this small dataset alone.
- Use feature-importance plots as model diagnostics, not as causal explanations.

6. Experiment Tracking and Reproducibility

The training script creates an MLflow experiment named `heart-disease-classification`, with one parent run per model family and a nested child run for every GridSearchCV parameter configuration. Child runs log the candidate hyperparameters and the mean/standard deviation for validation ROC-AUC, accuracy, precision, and recall. Each parent run includes selected hyperparameters, split/CV settings, final holdout metrics, a fitted scikit-learn pipeline, JSON evaluation summary, classification report, plot artifacts, and the complete CV candidate-results table.

Artifact	Purpose
`models/heart_disease_model.joblib`	Serializable preprocessing + classifier pipeline used by the API.
`artifacts/*_cv_results.csv`	Per-configuration mean/std train-fold metrics and hyperparameters.
`artifacts/*_classification_report.json`	Precision, recall, F1 and support by target class.
`artifacts/model_metadata.json`	Selected model family, feature order and target mapping.
`reports/metrics.json`	Consolidated model metrics, selection protocol and limitations.
`data/processed/*.manifest.json`	Dataset provenance, source and SHA-256 digest.

Reproduce the run

Create a clean Python 3.11 virtual environment, install `requirements.txt`, install the project with `pip install -e .`, download the data, then run the EDA and training module commands in the README. `random_state=42` is used for splitting, CV shuffling, and estimator seeds where applicable. Exact metric values can still vary across library versions or numerical backends, so the installed environment and final output files should be retained.

Run `mlflow ui --backend-store-uri ./mlruns --host 127.0.0.1 --port 5000` to inspect the local tracking store. The default store is local for the demonstration; a shared remote tracking server and artifact store would be required for multi-developer production use.

7. Automated Testing, CI/CD, and Containerization

Testing and linting

Ruff checks style and common defects. Pytest covers target binarization, missing-value preservation, pipeline fitting and probability output, API health and Prometheus metrics, prediction payload validation, and successful inference using a small deterministic test model. The test suite does not rely on external dataset access, allowing quick deterministic feedback on a laptop or CI runner.

GitHub Actions flow

Stage	Behavior / failure condition
Checkout + Python 3.11	Starts with a clean runner and pip install from requirements.
Ruff	Workflow fails if selected lint rules report errors.
Pytest	Workflow fails if a unit or API test fails.
Dataset + EDA	Workflow downloads official data then creates plots/summary.
Train + tracking	Tuning, held-out evaluation and MLflow artifacts run after tests.
Upload artifacts	Stores reports, model/evaluation artifacts and processed CSV for 14 days.

Docker packaging

The Dockerfile uses Python 3.11 slim and a separate `requirements-api.txt` to avoid installing MLflow and developer tooling in the serving image. It copies the service source and the trained joblib artifact, sets `MODEL_PATH`, exposes port 8000, and includes a health check. Train the model first; the Docker build deliberately fails if the expected artifact does not exist.

Run `docker compose up --build -d`; verify the API docs at `http://localhost:8000/docs`, then check `docker compose ps` and logs. Never put credentials, private data, or a production patient record in source control or image layers.

8. Deployment, Logging, and Monitoring

API contract

Endpoint	Purpose
`GET /health`	Process liveness; does not imply a trained model is present.
`GET /ready`	Readiness check for the model artifact.
`POST /predict`	Validates all 13 fields and returns class, class-1 probability and confidence.
`GET /metrics`	Prometheus counters and request latency histogram.

Operational metrics

- Request counter labelled by method, path, and HTTP status code.
- Latency histogram supporting p95 latency calculation.
- Prediction counter split by predicted class, without recording patient features.
- Request logs contain method/path/status/elapsed milliseconds, not the input body.

Docker Compose starts Prometheus and Grafana and provisions a dashboard for request rate, p95 latency, prediction counts and HTTP 5xx rates. Generate sample traffic from Swagger or the supplied client, wait at least two scrape intervals, and verify Prometheus's target status is UP. The dashboard is operational visibility, not data/model drift monitoring; no ground-truth feedback loop exists in this static demo.

Kubernetes

`k8s/deployment.yaml` defines two API replicas, resource requests/limits, readiness and liveness probes, scrape annotations, and a LoadBalancer Service. For Minikube, load the image into the cluster and use `kubectl port-forward svc/heart-disease-api 8000:80`. A real external LoadBalancer depends on support in the target environment; a manifest alone does not prove public cloud deployment.

Production gaps

Before any non-demo deployment, add HTTPS/TLS, authentication and authorization, rate limiting, network restrictions, secret management, audit controls, privacy review, model/version registry, alerting, monitoring of input drift and observed outcomes, and governance approvals. This health-related dataset is not clinically valid for decision support.

9. Demo Runbook, Rubric Mapping, and References

A complete demonstration should be recorded from a clean environment and should show the outputs rather than only the source code. Execute the following commands from the repository root in the order shown.

Step	Command / evidence
1. Setup	`pip install -r requirements.txt` then `pip install -e .`
2. Acquire data	`python -m heart_mlops.data --output data/processed/heart_disease_clean.csv`
3. EDA	`python -m heart_mlops.eda --input data/processed/heart_disease_clean.csv --output reports/figures`
4. Train	`python -m heart_mlops.train --input data/processed/heart_disease_clean.csv --artifacts-dir artifacts --reports-dir reports --mlflow-dir mlruns`
5. Report	`python scripts/generate_report.py` (reads measured metrics when present)
6. Quality gates	`ruff check src tests scripts` and `pytest`
7. API	`uvicorn heart_mlops.api:app --reload --port 8000`; test `/docs`, `/predict`, `/metrics`
8. Monitor	`docker compose up --build -d`; capture Prometheus and Grafana screenshots
9. Deploy/CI	Push to GitHub to run Actions; optionally apply the Minikube manifest and capture `kubectl` output.

Assignment deliverable map

EDA and data download are implemented in `data.py` and `eda.py`; feature engineering, two models and CV tuning are in `modeling.py` and `train.py`; MLflow logs experiments; joblib + requirements provide a reusable model and environment; pytest/Ruff/GitHub Actions cover automated quality checks; Docker and Kubernetes files implement packaging/deployment; request logs and Prometheus/Grafana cover monitoring; this report plus architecture diagram cover documentation. Record your actual repository URL and public API URL only after creating/validating them.

References

[1] UCI Machine Learning Repository, Heart Disease, https://archive.ics.uci.edu/dataset/45/heart+disease (dataset description, class definition, missing values and CC BY 4.0 attribution).

[2] MLflow tracking/autologging, https://mlflow.org/docs/latest/ml/tracking/autolog/ . [3] FastAPI testing, https://fastapi.tiangolo.com/tutorial/testing/ .

[4] Prometheus Python client, https://prometheus.github.io/client_python/ . [5] Kubernetes external LoadBalancer services, https://kubernetes.io/docs/tasks/access-application-cluster/create-external-load-balancer/ .

Final evidence checklist: add your GitHub repository URL, replace metrics only with values from the real run, save EDA/MLflow/Swagger/Docker/Grafana/GitHub Actions/Kubernetes screenshots as relevant, and include a short screen recording of the end-to-end flow.